UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
Índice de Contenidos

- 1 – Introducción
- 2 – Versiones ECMAScript
- 3 – De Java a JavaScript
- 4 – Entorno de trabajo y primer script
- 5 – Elementos básicos del lenguaje
- 6 – El navegador como entorno de ejecución
- 7 – Integrando JS con HTML
- 8 – Selección de elementos del DOM con JS
- 9 - Manipulación de los nodos del DOM con JS
- 10 – Vanilla JS vs Frameworks/Librerías
- Anexo – Depurando JS desde el navegador

1 - Introducción

- Un poco de historia. Mediados de los 90. Internet entra en los hogares y tiene una gran proyección de mercado. Las páginas web son "feas", CSS está naciendo y HTML está en la versión 2.0.
- Es la época de la "**guerra de los navegadores**". Internet Explorer y Netscape luchan por ofrecer la mejor experiencia de navegación al usuario para ganar cuota de mercado.
- Java quiere su parte del pastel y crea los "**applets**" una tecnología que permite ejecutar programas en el navegador, pero que es lenta y requiere la instalación de plugins…



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
1 – Introducción

- JavaScript es desarrollado por Netscape a finales de 1995 para dotar de **dinamismo** y de **interactividad** a las páginas web de la época. Es un lenguaje **interpretado** y ejecutado en el navegador.
  - *Inicialmente se iba a llamar **LiveScript**, pero se quiso aprovechar el tirón mediático de Java y se le llamó JavaScript.*
- Se **crea en tiempo "record"** (10 días) y pretende ser un lenguaje que sea **fácil de aprender** por "todo el mundo". Estas dos circunstancias hacen que sea un lenguaje bastante peculiar y que arrastra errores de diseño desde sus inicios.
  - Tiene el "honor" de ser simultáneamente el lenguaje más amado y odiado por la comunidad de desarrolladores.
  - Aprender JS implica también aprender a quedarte las "partes buenas" del lenguaje y esquivar las "partes malas"…



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
1 – Introducción

Alternativas a JS

- JS es el lenguaje de script del lado cliente más extendido. Casi sin competidores, aunque alguno hay:
  - **Typescript (TS):** es un superconjunto de JS, creado por Microsoft, que aporta el tipado estático y algunas mejoras, permitiendo crear un código más robusto y fiable.

    También permite una mejor organización del código mediante la creación de módulos, clases, interfaces…

    Sin embargo, **TS se "transpila" a JS**, es decir, se compila a código fuente JS. Por este motivo se podría decir que, más que una alternativa "real" a JS, es una mejora.



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
1 – Introducción

- **WebAssembly (WA):** es un estándar abierto desarrollado por Mozilla, Google, Microsoft y Apple. WA **define el conjunto de instrucciones** que se pueden ejecutar en la máquina virtual que llevan los navegadores. No es un lenguaje de programación.

  Este enfoque permite una **ejecución más eficiente** del código en el navegador, beneficiando las aplicaciones exigentes en términos de computación (video-juegos, herramientas de edición de vídeo o de audio…)

  WA se integra con muchos lenguajes de programación y permite utilizar las librerías y herramientas existentes.



WEBASSEMBLY

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
2 – Versiones ECMAScript

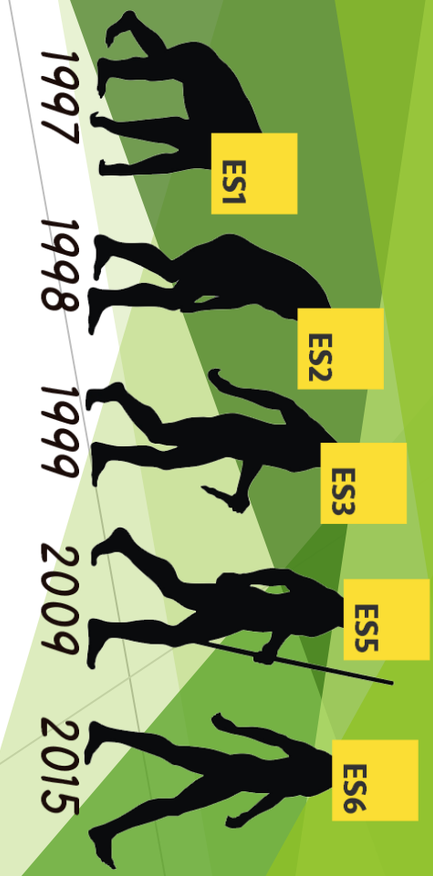
2 – Versiones ECMAScript

- JS fue ganando fama desde que nació y los distintos navegadores desarrollaban sus propios intérpretes del lenguaje. Esto provocó "**divergencias en la ejecución**" y que un mismo código se podía comportar de forma distinta dependiendo del navegador que lo ejecutase.
- La ECMA (European Computer Manufacturers Association) decide intervenir y crear **ECMAScript** que es una especificación de cómo debe interpretarse el código JS.
- Técnicamente JavaScript y ECMAScript son dos cosas distintas, pero coloquialmente son términos intercambiables.



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
2 – Versiones ECMAScript

- Las especificaciones de ECMAScript (ES) han ido pasando por distintas versiones:
  1. **ECMAScript 1 (ES1)**: se lanzó en 1997, fue la primera versión estandarizada de JS que estableció las bases del lenguaje y de cómo debía ejecutarse. Incluyó características fundamentales como variables, funciones y objetos.
  2. **ECMAScript 2 (ES2)**: lanzada en 1998, esta versión introdujo mejoras menores, como soporte para expresiones regulares y modificadores de variables.
  3. **ECMAScript 3 (ES3)**: ES3 se lanzó en 1999 y fue una versión importante que estableció la base para el desarrollo web moderno. Introdujo nuevas características como manejo de excepciones, los métodos de los arrays y la notación JSON.



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
2 – Versiones ECMAScript

4. **ECMAScript 4 (ES4)**: aunque se trabajó en esta versión, nunca fue lanzada oficialmente debido a desacuerdos en la comunidad. Las propuestas de ES4 fueron consideradas demasiado radicales y se decidió realizar cambios incrementales en futuras versiones.
5. **ECMAScript 5 (ES5)**: publicada en 2009, ES5 fue una versión importante para estandarizar las características existentes de JavaScript. Introdujo mejoras como el soporte para getters y setters, manejo estricto de variables ("strict mode") y más.
6. **ECMAScript 6 (ES6) / ECMAScript 2015**: Lanzada en 2015, ES6 fue una actualización significativa que trajo muchas características nuevas. Introdujo conceptos como clases, módulos, arrow functions, destructuring, promesas, entre otros. Esta es la versión que vamos a estudiar en este curso.



3 – De Java a JavaScript

- Java y Javascript (JS) son dos lenguajes **DISTINTOS**, solo que por razones históricas/comerciales, ambos comienzan por la palabra "Java".
- Eso significa que para aprender JS tendremos que "abandonar" ciertos conceptos que funcionan en Java y no "forzar" a que un lenguaje encaje en el otro.
- Antes de entrar en los detalles de JS, hagamos una comparativa de ambos lenguajes para ver comprender mejor sus diferencias.



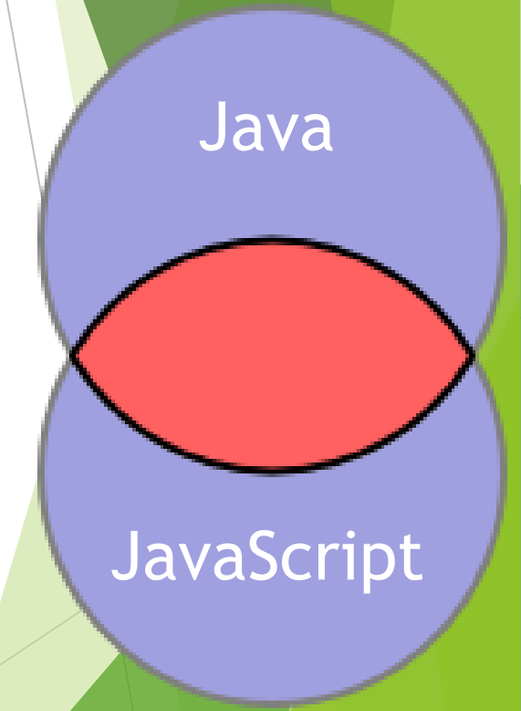
UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
3 – De Java a JavaScript

- Comparativa:

JAVA	JAVASCRIPT
Sigue el paradigma Orientado a Objetos puro.	Sigue un paradigma OO "propio" que no está basado en la herencia de clases, sino en prototipos.
Existen modificadores de acceso: private, public…	No existen modificadores de acceso a los miembros de un objeto. Todo se considera public.
No permite el paradigma Estructurado. Los métodos siempre se presentan dentro de una clase.	Permite el paradigma Estructurado en el que las funciones son métodos "sueltos", que no están incluidos en ninguna clase.
No es un lenguaje de script y para ejecutar un programa hace falta una estructura concreta: método main.	Es un lenguaje de script, así que para lanzar un script solo hace falta una sucesión de instrucciones, sin apenas estructura.
Es de tipado fuerte (si una variable es de un tipo será de ese tipo durante todo el programa)	Es de tipado débil (las variables pueden cambiar de tipo durante el programa)
El compilador es "estricto" con el programador y no hace suposiciones, si hay un error te lo señala.	El intérprete es "amable" con el programador y hace algunas suposiciones para "pasar por alto" algunos errores.
Su entorno de ejecución es la MVJ que puede "estar sola" o embebida en un navegador.	Su entorno de ejecución "natural" es el intérprete de JS que tienen los navegadores, aunque también hay intérpretes "en solitario" como Node.JS.

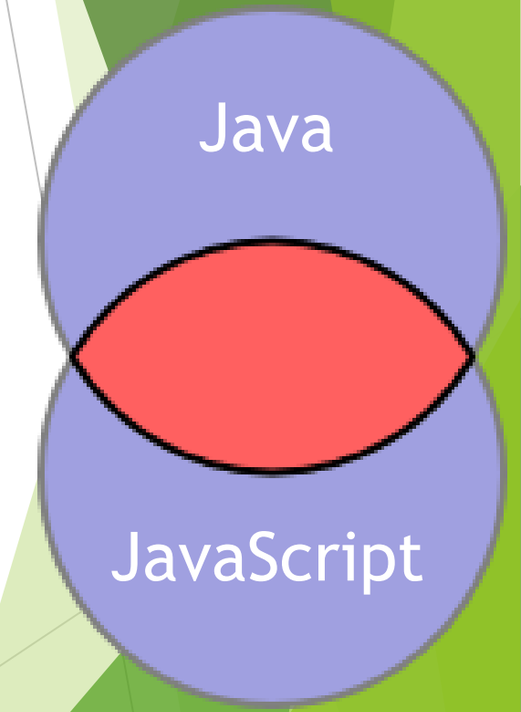
UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
3 – De Java a JavaScript

- Pero también hay muchos conceptos y **elementos** sintácticos **comunes** que funcionan exactamente igual en ambos lenguajes:
  - Comentarios con // o con /* … */
  - Las instrucciones terminan con ; (aunque los navegadores aceptan que se te olvide)
  - Se consideran identificadores válidos (para nombres de funciones o variables…) aquellos que comienza por letra, _ o $ y le siguen otras letras, números o _ o $
  - El estilo de nombrado también sigue las mismas reglas que en Java (UpperCamelCase para Clases/Prototipos y lowerCamelCase para el resto de los elementos del lenguaje)



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
3 – De Java a JavaScript

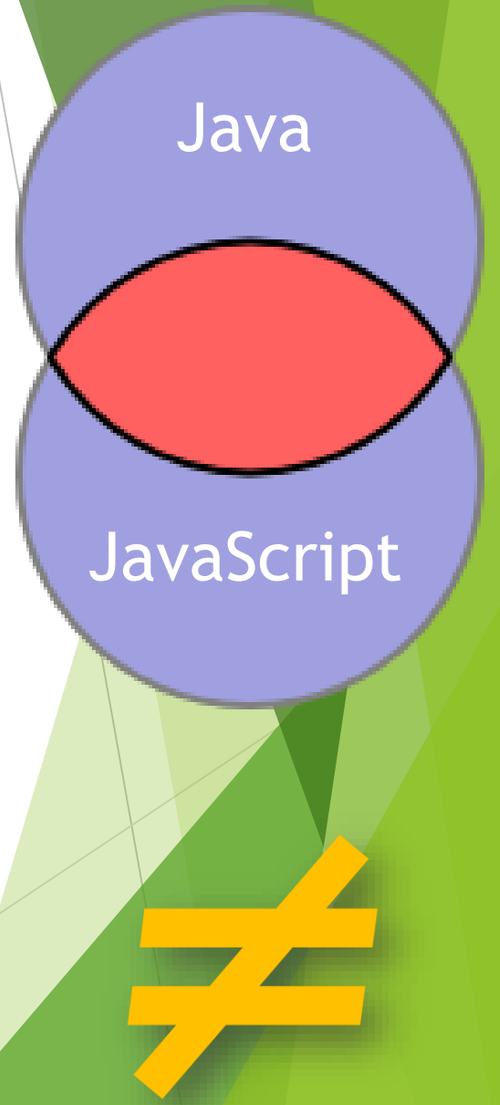
- Mismas reglas de indentación
- Condicionales: if, if…else y switch
- Condicional ternaria: condición ? valor1 : valor2;
- Bucles: while, do…while y for
- Operadores de asignación: =, +=, -= …
- Operadores aritméticos: +, -, *, /, %...
- Operadores de autoincremento/decremento: ++, --
- Operadores de comparación: ==, !=, >=, <=… (aunque el == funciona un poco diferente). Además, Javascript añade otro operador de comparación extra: ===



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
3 – De Java a JavaScript

- Operadores lógicos: &&, || y !
- Operador de concatenación de cadenas: +
- Operador de acceso a un miembro de un objeto: .
- Orden de prioridad de los operadores
- Invocación de funciones/métodos:

  nombreDeLaFuncion(argumento1, argumento2)

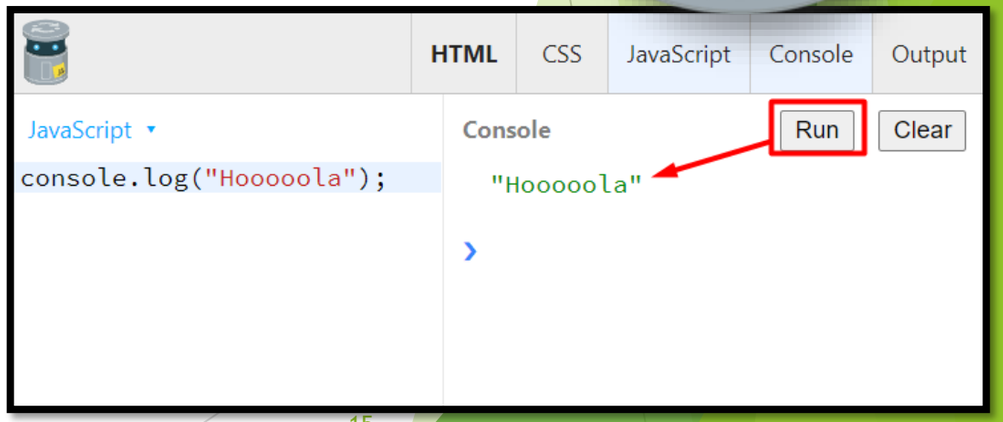


Eso sí… *"no es ORO todo lo que reluce"*. La forma de trabajar con los objetos y prototipos de JS es MUY DIFERENTE a cómo se realiza en Java…

4 - Entornos de desarrollo y primer script

- En apartado 7 de esta unidad integraremos JS con HTML y CSS y utilizaremos la pareja VSCode + Navegador como entorno de desarrollo.
- Sin embargo, en esta primera parte, estudiaremos JS con independencia de HTML/CSS. Así que utilizaremos uno de los **entornos web de experimentación** disponibles en Internet que nos permitirán ver simultáneamente el código y la consola.
- **JS Bin** (papelera JS) tiene esta pinta
  - Se accede desde la web: https://jsbin.com/?js,console
  - Usaremos las pestañas "JavaScript" y "Console".
  - Con el botón "Run" ejecutamos el script de JS.
  - Con el botón "Clear" se limpian los mensajes de la consola.





UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
4 – Entornos de desarrollo y primer script

- Nuestro primer programa simplemente escribirá un mensaje en la consola.



- ¡Ea! Ya sabemos escribir en la consola de JS. **console** es un objeto que definen todos los navegadores y **log** es un método de este objeto.
- Como vemos, hemos lanzado un **script**, es decir, una secuencia de instrucciones. No tenemos un *main*, ni estamos obligados a declarar clases, ni debemos seguir una estructura concreta, simplemente escribimos las instrucciones que queremos que se ejecuten



5 - Elementos básicos del lenguaje

Declaración de variables y constantes

- Observa el siguiente script y su resultado en consola.

```
JavaScript ▾
let edad = 33;
let nombre = "Pepe";
console.log("Hola "+nombre+" tienes "+edad+" años");
```

```
Console                    Run   Clear
  "Hola Pepe tienes 33 años"
›
```

- Como vemos las variables se declaran con **let** ("permite" que se use la variable X) **Y NO HAY QUE PONER EL TIPO**. El tipo se "deduce" directamente de la asignación.

¿Y qué pasa si no le asigno nada a la variable?



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

JavaScript

```
let edad;
let nombre = "Pepe";
console.log("Hola "+nombre+" tienes "+edad+" años");
```

Console

```
"Hola Pepe tienes undefined años"
```

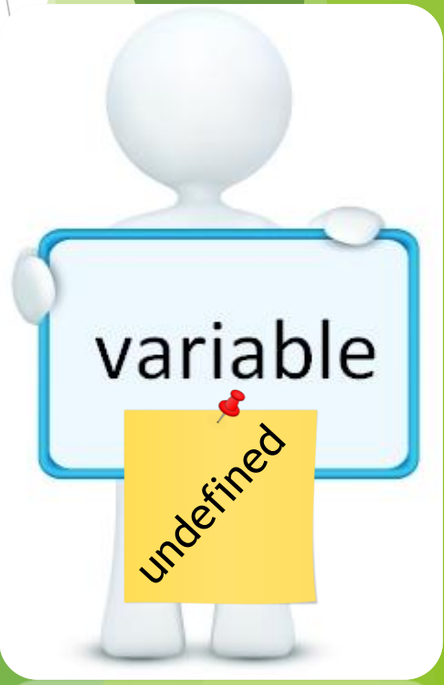
- Como vemos las variables no inicializadas están "**marcadas**" como "undefined" y así aparecerán cada vez que queramos mostrarlas en la consola.
- En cuanto se les asigna un valor, se les quita esta marca.

JavaScript

```
let edad;
let nombre = "Pepe";
console.log("Hola "+nombre+" tienes "+edad+" años");
edad = 21;
console.log("Hola "+nombre+" tienes "+edad+" años");
```

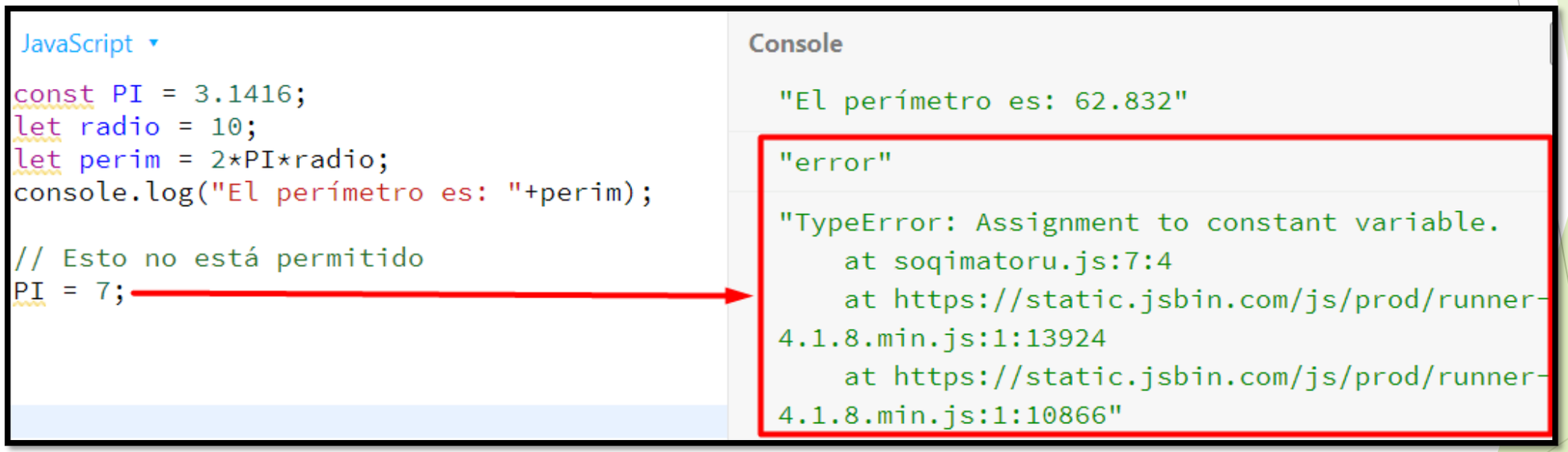
Console

```
"Hola Pepe tienes undefined años"
"Hola Pepe tienes 21 años"
```



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Las constantes se definen con **const** y una vez se les asigne un valor no se podrán modificar.



Realiza los ejercicios 1 y 2 del boletín de problemas

- Hay otra forma de declarar variables que usa **var,** en vez de **let,** pero hoy se considera una **mala práctica** (da problemas de "ámbito"). Lo veremos en detalle un poco más adelante.



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

Los tipos de datos en JS

- Al igual que en Java, JS clasifica los tipos de datos en dos grandes grupos:
  - **Tipos primitivos**: son tipos simples que sirven para representar un valor concreto de una magnitud (3, true, 4.5…). No son objetos, no tienen métodos ni propiedades.
    - Los estudiamos en la siguiente diapositiva.
  - **Tipos objeto:** representan una realidad más compleja que normalmente forma parte del problema que se quiere resolver (Factura, Nómina, Persona…). Poseen propiedades y métodos. Al igual que en Java, se usan referencias para "apuntar" a los objetos que se creen en JS.
    - La creación y utilización de objetos en JS se trabajará profundamente en el 2º curso de este ciclo.





UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

Los tipos primitivos en JS son:

- **number**: es el equivalente al double en Java. Es un número real que se almacena en 8 bytes. En JS no existe ni int ni integer. **Todos los números son number**.
- **string**: sirve para representar textos con caracteres Unicode de 16 bits. En JS no existe el tipo char. Si queremos representar un solo carácter usaremos una cadena para ello. String en Java es un objeto, pero string en JS no.
- **boolean**: define los valores true y false. Funciona igual que en Java.
- **undefined**: es un tipo nuevo que define un solo valor llamado "undefined". Como ya vimos, este valor se usa para "marcar" a una variable que aún no se le ha asignado ningún valor.
- **null**: es un tipo que solo define el valor "null" y se usa para "marcar" que una referencia a un objeto aún no ha sido inicializada.



Observa que todos empiezan con minúscula ya que NO SON OBJETOS

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

JS es un lenguaje de tipado débil

Cuando hablamos del "**tipado**" de un lenguaje nos referimos a la forma en la que gestiona el tipo de las variables y expresiones.

Java es un lenguaje de **"tipado fuerte"** porque nos obliga a definir el tipo de una variable cuando la declaramos, entonces **el tipo queda "fijado"** durante todo el programa. Además, no podemos hacer operaciones que mezclen distintos tipos.

Por ejemplo, esto en Java se considera un error porque la variable *mayorEdad* está declarada como boolean así que no puede almacenar un entero:

```
boolean mayorEdad = true;
mayorEdad = 18;
```



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

JS es un lenguaje de **"tipado débil"** y se permite que las variables puedan cambiar su tipo durante la ejecución del programa. Observa:

Tipo number

Tipo boolean

```
JavaScript
let mayorEdad = true;
console.log ("Valor de la variable = "+mayorEdad);
mayorEdad = 18;
console.log ("Valor de la variable = "+mayorEdad);
```

```
Console
"Valor de la variable = true"
"Valor de la variable = 18"
```

Con este nuevo enfoque **las variables no tienen un tipo fijo** o se podría decir que ahora son de **tipo "mixto"**. Solo podremos afirmar que en un instante dado una variable contiene un valor cuyo tipo es X. Esto permite una sintaxis más libre y expresiva.

Sin embargo, JS además hace **"conversiones automáticas de tipo"** en cualquier tipo de expresión. Esto abre una "caja de pandora" muy peligrosa puesto que un error del programador puede derivar en **resultados inesperados** de todo tipo. Veámoslo.



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

Conversión automática de tipos (type coercion)

- Observa:

"hola" se convierte automáticamente a boolean con valor true

```
JavaScript
if ("hola" && 3 < 10) {
  console.log("Cierto");
}
```

```
Console
  "Cierto"
>
```

- Como vemos JS hace "conversiones automáticas" de los tipos de una expresión "forzándolos" para que todos se adapten a un tipo de dato.
- Estas conversiones automáticas se pensaron para **"hacer la vida más fácil"** al programador y pretendían solucionar cosas como esta:

```
JavaScript
let edadActual = "10";
edadPasada = edadActual - 1;
console.log("Tienes "+edadActual+" años y el año pasado tenías "+edadPasada);
```

```
Console
  "Tienes 10 años y el año pasado tenías 9"
>
```


JS convierte automáticamente el valor "10" a **number** para poder restarle 1. ¡BIEN!

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Sin embargo, también nos puede dar errores inesperados como este:

```
JavaScript
let edadActual = "10";
edadNueva = edadActual + 1;
console.log("Tienes "+edadActual+" años y el próximo año tendrás "+edadNueva);
```

```
Console
"Tienes 10 años y el próximo año tendrás 101"
>
```

JS convierte automáticamente el valor 1 a **string** para poder concatenarlo con "10". ¡MAL!

- Veamos algunos ejemplos extraños más:

"123" + 4 se evaluará como "1234"

"123" – 4 se evaluará como 119

"1" – "1" se evaluará como 0

34 + true se evaluará como 35

2 * false se evaluará como 0

-true se evaluará como -1

"true" + true se evaluará como "truetrue"

Para saber más: https://www.escuelafrontend.com/coercion-de-datos-en-javascript

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

Comparando valores con == y ===

- JS sigue sorprendiéndonos:

JavaScript

```
if ("1" == 1) {
  console.log("Son iguales");
}
```

Console

```
"Son iguales"
>
```

En Java esto provocaría un error sintáctico

- Con el operador == primero se realiza una **conversión automática de tipos** y después se comparan los valores resultantes.
- Este comportamiento facilita que se produzcan **errores difíciles de detectar**. Hoy en día se considera una mala práctica usar == y !=
- Más ejemplos de resultados inesperados:

Expresiones con == y !=
if ("123" == 123) devuelve *true*
if (0 == false) devuelve *true*
if ("" == false) devuelve *true*



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Para solucionarlo, JS proporciona dos operadores más: **=== y !==**
- Estos operadores devuelven false si **los operandos tienen tipos distintos**. En caso de que ambos sean del mismo tipo, se realiza la comparación y se devuelve el resultado que corresponda.
- Se considera buena práctica usar los operadores === y !==.
- Por cierto, las cadenas no se comparan con el método *equals* como en Java. También se comparan usando === y !==. Observa:

Con === se arreglan los problemas anteriores

Expresiones con === y !==
if ("123" === 123) devuelve *false*
if (0 === false) devuelve *false*
if ("" === false) devuelve *false*

JavaScript

```
let a = "hola";
let b = "hola";

if (a === b) {
   console.log("Iguales");
}
```

Console

```
"Iguales"

>
```

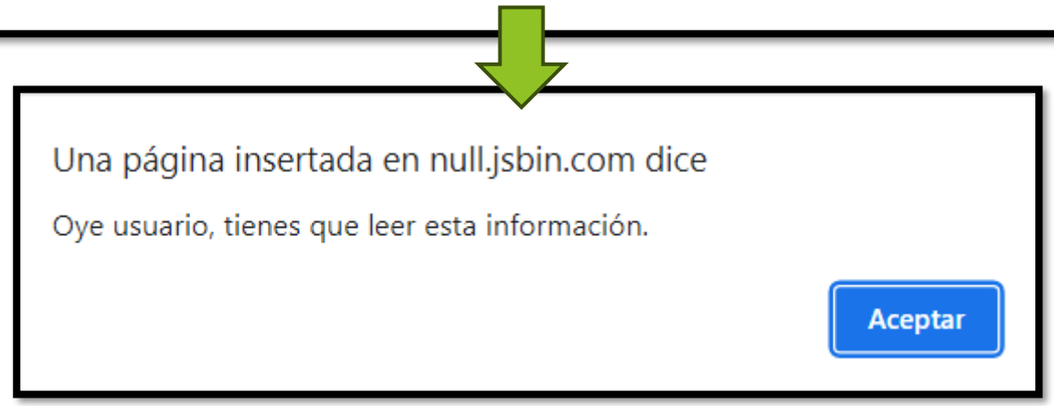


UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

Ventanas de alerta

- La consola del navegador no suele mirarla el usuario estándar… Cuando queremos **llamar la atención del usuario** podemos abrir una "ventana de diálogo" que le fuerce a leer cierta información y pinchar en un botón para poder continuar
- La sintaxis es muy sencilla. Observa:

```
alert("Oye usuario, tienes que leer esta información.");
```





UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

Ventana de confirmación

- Si queremos que el usuario nos confirme una acción y necesitamos **llamar su atención y obtener una respuesta booleana** podemos usar una "ventana de confirmación".
- La sintaxis es muy sencilla. Observa:

```
let respuesta = confirm("¿Estás seguro de eliminar el producto?");

console.log(respuesta);
```





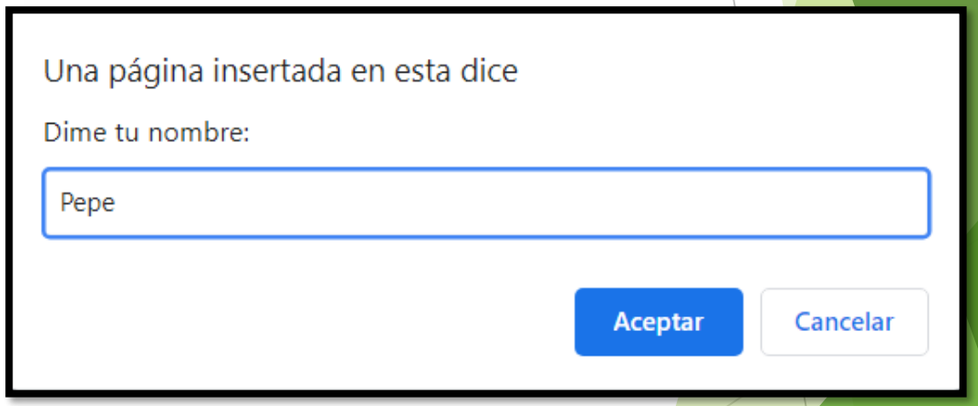
UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

Entrada por teclado

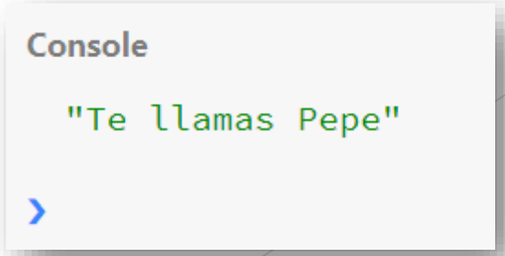
- Observa el siguiente trozo de código:

```
JavaScript ▾
let nombre = prompt("Dime tu nombre: ");
console.log("Te llamas "+nombre)
```

- Con **prompt** pedimos al navegador que muestre una ventana en la que puede teclear el usuario.



- El resultado tecleado se guarda en la variable nombre y se muestra en la consola:



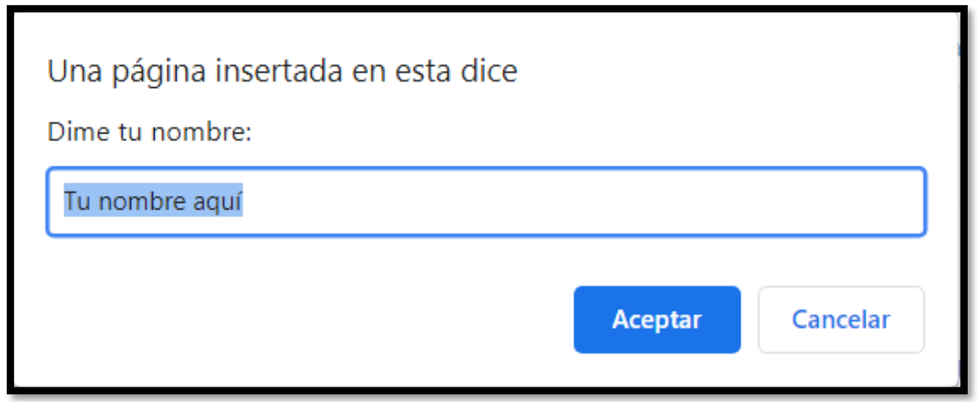
UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Si pinchamos en el botón "Cancelar" obtendríamos:

```
Console
  "Te llamas null"
>
```

- También podemos poner un parámetro más que actúa como **"texto de ejemplo"** en la ventana:

```
JavaScript ▾
let nombre = prompt("Dime tu nombre: ", "Tu nombre aquí");
console.log("Te llamas "+nombre);
```



Realiza el ejercicio 3 del boletín de problemas

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Observa también lo siguiente:

```javascript
let edad = prompt("Dime tu edad: ");
console.log("Hoy tienes "+edad+" años");
let edadNueva = edad + 1;
console.log("El año que viene tendrás "+edadNueva+" años");
```

```
Console
"Hoy tienes 32 años"
"El año que viene tendrás 321 años"
```

- Esto se produce porque **prompt** devueve un valor de tipo **string**. Para forzar el cambio de la cadena de texto a un número deberíamos usar Number, del siguiente modo:

Number es un objeto "wrapper" que actúa como envoltorio del tipo primitivo number.

```javascript
let edad = Number(prompt("Dime tu edad: "));
console.log("Hoy tienes "+edad+" años");
let edadNueva = edad + 1;
console.log("El año que viene tendrás "+edadNueva+" años");
```

```
Console
"Hoy tienes 32 años"
"El año que viene tendrás 33 años"
```

Realiza los ejercicios 4-5 del boletín de problemas

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

El operador typeof

- Hay veces que necesitamos asegurarnos de que una variable tiene el tipo adecuado para realizar una operación. Para ello podemos hacer uso del operador **typeof.** Observa:

```
JavaScript ▾
let valor = true;

if(typeof valor === "number") {
  console.log("Es un número");
}

if(typeof valor === "string") {
  console.log("Es una cadena");
}

if(typeof valor === "boolean") {
  console.log("Es un boolean");
}
```

```
Console
  "Es un boolean"
›
```

Observa que typeof devuelve una cadena con el tipo

- También podemos usar este operador con expresiones:

```
console.log(typeof (3+3.3));
console.log(typeof ("hola"));
console.log(typeof (3 < 4));
```

```
"number"
"string"
"boolean"
```

OJO: si quitamos los paréntesis que encierran la expresión, pueden pasar cosas "raras"

```
"number3.3"
"string"
false
```

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

Funciones

- Las funciones son **piezas de código aisladas que resuelven un problema** concreto y que pueden ser invocadas desde otras partes del programa, pudiendo así reutilizarse. Son como los métodos de un objeto, pero sacados fuera del objeto.
- Las funciones deben ser "declaradas" antes de ser utilizadas. Observa la sintaxis de la declaración:

JavaScript

```
// Declaración. Observa que los parámetros de entrada de la función
// se escriben directamente, sin "let" ni tipos ni nada.
function muestraSuma (num1, num2) {
    let suma = num1+num2;
    console.log ("La suma es "+suma);
}
```

Console

›

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Sin embargo, el código anterior solo "declara" la función, pero no la ejecuta. Para ello debemos "invocar o llamar" a la función dándole los parámetros de entrada que necesite. Observa:

JavaScript

```
// Declaración. Observa que los parámetros de entrada de la función
// se escriben directamente, sin "let" ni tipos ni nada.
function muestraSuma (num1, num2) {
    let suma = num1+num2;
    console.log ("La suma es "+suma);
}

let a = 10, b = 20;

// Llamada a la función
muestraSuma(a, b);
```

Console

```
"La suma es 30"

>
```

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Dado que no se especifica el tipo de los parámetros de entrada, nuestro script puede **hacer cosas raras dependiendo de los parámetros que se le pase**. Observa:

JavaScript

```
// Declaración. Observa que los parámetros de entrada de la función
// se escriben directamente, sin "let" ni tipos ni nada.
function muestraSuma (num1, num2) {
    let suma = num1+num2;
    console.log ("La suma es "+suma);
}

let a = "hola", b = "adios";

// Llamada a la función
muestraSuma(a, b);
muestraSuma("1", "2");
```

Console

```
"La suma es holaadios"
"La suma es 12"
```



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Observa también que a JS no le parece mal si invocas una función con más o menos parámetros de los declarados. Simplemente lo ejecuta y que salga "lo que Dios quiera":

JavaScript

```
function muestraSuma(a, b, c) {
  let suma = a + b + c;
  console.log(suma);
}

muestraSuma(2, 3, 4, 4, 5, 5, 4);
muestraSuma(1);
muestraSuma("1", "2");
```

Console

```
9
NaN
"12undefined"
```

›

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Si queremos que nuestra función calcule un dato y lo devuelva solo tenemos que añadir la instrucción **return**. Observa:

JavaScript ▾

```
function calculaSuma(num1, num2) {
  let suma = num1 + num2;
  return suma;
}

let a = 10, b = 20;

console.log("La suma de "+a+" y "+b+" es "+calculaSuma(a, b));
```

Console

```
"La suma de 10 y 20 es 30"
```

- También podríamos haber "recogido" el valor devuelto en una variable:

```
let a = 10, b = 20;

let suma = calculaSuma(a,b);
console.log("La suma de "+a+" y "+b+" es "+suma);
```

Realiza los ejercicios 6-7 del boletín de problemas

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

let VS var

- Una variable declarada con **let** funciona igual que en Java, es decir, tiene "validez y existencia" dentro del ámbito { } en el que fue declarada.

```
function muestraSuma(num1, num2) {
  if(num1 < num2) {
      let suma = num1 + num2;
  }

  return suma;
}

let a = 10, b = 20;

console.log(muestraSuma(a, b));
```

```
"error"

"ReferenceError: suma is not defined
    at muestraSuma (kenepeboku.js:6:3)
    at kenepeboku.js:11:38
    at https://static.jsbin.com/js/prod/runner-
4.1.8.min.js:1:13924
    at https://static.jsbin.com/js/prod/runner-
4.1.8.min.js:1:10866"
```



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Sin embargo, el ámbito de una variable declarada con **var** es **todo el código de la función que la contiene**.

```
function muestraSuma(num1, num2) {
  if(num1 < num2) {
      var suma = num1 + num2;
  }

  return suma;
}

let a = 10, b = 20;

console.log(muestraSuma(a, b));
```



```
Console

30
```

Para saber más:
https://codingpotions.com/javascript-diferencia-var-let-const/

- **¿Y qué pasaría si ejecutamos los siguiente?**

```
console.log(muestraSuma(20, 10));
```



```
Console

undefined
```

Conclusión rápida:
usar **var** puede dar problemas.
Mejor usa let y const

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

Objetos en JS: lo básico

- Java sigue una filosofía "**industrial**" respecto a la creación de objetos. Es decir, todo objeto **se crea con un molde**, la clase.
- Sin embargo, JS sigue una filosofía más "**artesanal**" y **no es necesario tener un molde (clase) para crear un objeto**. Es decir, JS nos permite crear un objeto **agrupando un conjunto de propiedades y métodos**. Veamos un ejemplo:

```
let user = {
  name: "Juan",
  age: 30
};

console.log(user);
console.log("El valor de la propiedad name es "+user.name);
```

Se le llama "notación literal" a esta forma de crear objetos describiéndolos como una lista de "propiedad: valor"



Creación "industrial"

VS

Creación "artesanal"



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- El código anterior imprimiría en la consola lo siguiente:

Nos indica que es un objeto que hereda de Object

```
[object Object] {
  age: 30,
  name: "Juan"
}
```

Las propiedades son siempre PÚBLICAS

```
"El valor de la propiedad name es Juan"
```

- Si queremos añadir **métodos** lo hacemos igual que con las propiedades:

```
let user = {
  name: "Juan",
  age: 30,
  print: function() {
    console.log("Usuario "+this.name+" de "+this.age+" años");
  }
};

user.print();
```

this es obligatorio, si no se pone nos da un error



```
"Usuario Juan de 30 años"
```

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Hay una **notación abreviada** equivalente a la anterior a la hora de crear métodos:

```
let user = {
  name: "Juan",
  age: 30,
  print() {
    console.log("Usuario "+this.name+" de "+this.age+" años");
  }
};
```

- También podemos **cambiar los valores** de las propiedades y métodos asignándoles un nuevo valor:

```
user.age = 40;
user.print = function() { console.log("Edad = "+this.age);};

user.print();
```



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Podemos usar **nombres de propiedades que contengan espacios** escribiéndolos entrecomillados.

```
let user = {
  'user name': "Juan",
  age: 23
}

console.log(user);
```



```
[object Object] {
  age: 23,
  user name: "Juan"
}
```

También se pueden encerrar entre comillas dobles "

- También podemos crear **propiedades con valores tipo objeto**:

```
let user = {
  name: "Juan",
  age: 23,
  address: { street: "de Juan", number: 11 }
}

console.log(user);
```



```
[object Object] {
  address: [object Object] {
    number: 11,
    street: "de Juan"
  },
  age: 23,
  name: "Juan"
}
```

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Además, la estructura de un objeto **no se bloquea cuando se crea**. **Siempre se pueden poner y quitar propiedades y métodos**. Observa:

```
let user = {}; // Creamos un objeto vacío

user.name = "Juan";
user.address = "Calle de Juan";
user.age = 23;
delete user.address;
user.print = function() {
  console.log(this.name+" : "+this.age+" años");
}

user.print();
console.log(user);
```

```
"Juan : 23 años"

[object Object] {
  age: 23,
  name: "Juan",
  print: function() {
  window.runnerWindow.proxyConsole.log(this.name+" :
"+this.age+" años");
}
}
```



Objetos con estructura FLEXIBLE

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Este comportamiento abre la puerta a **cometer errores tipográficos** y que se creen propiedades incorrectas "involuntariamente". Observa:

```
let user = {
  name : "Juan",
  address : "Calle de Juan",
  age : 23
};

user.naame = "Pedro";
user.age = 34;

console.log(user);
```

```
[object Object] {
  address: "Calle de Juan",
  age: 34,
  naame: "Pedro",
  name: "Juan"
}
```



- Además, hay que tener más cuidado todavía porque que el nombre de las propiedades es **sensible a mayúsculas y minúsculas**:

```
// continúación del ejemplo anterior
user.Age = 41;
console.log(user);
```

```
[object Object] {
  address: "Calle de Juan",
  age: 34,
  Age: 41,
  naame: "Pedro",
  name: "Juan"
}
```

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

Los objetos y la palabra const

- Es bastante habitual usar **const** con objetos. Siempre hemos usado **const** para declarar constantes: *const PI = 3.1416*. Sin embargo, su comportamiento cuando se usa con objetos es diferente. Observa:

```
const persona = { nombre: "Pedro", edad: 23};
persona.nombre = "Juan";
persona.direccion = "Dirección de Juan";
console.log(persona);
```

```
[object Object] {
  direccion: "Dirección de Juan",
  edad: 23,
  nombre: "Juan"
}
```

- Como vemos, sí que podemos modificar el objeto… Ya que **const impide que modifiquemos la referencia al objeto**. Observa:

```
persona = {name: "John"};
```

✖ "TypeError: Assignment to constant variable.

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Como vemos, el uso de los objetos en **JS y en Java difiere bastante**.
  - Los objetos JS ya **no necesitan una clase para crearse**, así que ya no tiene mucho sentido hablar de que un objeto JS es de "tipo X".
  - Además, su **estructura interna se puede cambiar** durante la ejecución.
  - La **notación literal** nos permite crear los objetos como queramos. Ya no hacen faltan obligatoriamente métodos constructores como en Java.
  - En muchas ocasiones los objetos JS no tendrán métodos y se comportarán como **simples "contenedores" de datos** que transfieren información de una parte a otra del script o la aplicación.
  - Por defecto los objetos no tienen getters ni setters y sus propiedades son públicas, dejando al **objeto "indefenso" ante posibles malos usos**. En Java, el objeto siempre debía "velar" por mantener la coherencia de su estado interno.



Realiza los ejercicios 8-9 del boletín de problemas

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

Arrays en JS: lo básico

- Los arrays en JS **se usan muuuuucho más** que en Java porque son **más versátiles y cómodos**.
- Los arrays son **objetos** y como tales tienen propiedades y métodos. Observa:

```
// Se crean así
let array = [10, 20, 30];
console.log("Tamaño del array = "+array.length);

// Podemos recorrerlos como siempre
for(let i = 0; i < array.length; i++) {
  console.log("Posición "+i+" = "+array[i]);
}

// Y también imprimirlos directamente
console.log(array);
```



```
"Tamaño del array = 3"
"Posición 0 = 10"
"Posición 1 = 20"
"Posición 2 = 30"
[10, 20, 30]
```

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- También podemos recorrerlos con un **for… of** (que es como el for… each):

```
let array = [ 10, 20, 30 ];

for (let elem of array) {
  console.log(elem);
}
```

```
10
20
30
```

- **Novedades con los límites** del array:

```
let array = [10, 20, 30];

// Si leemos fuera de los límites del array no explota
console.log(array[1000]);
console.log(array[-100]);

// Si escribimos fuera de los límites se redimensiona
// y los elementos nuevos se rellenan con undefined
array[10] = 100;
console.log(array);
```




```
undefined
undefined
[10, 20, 30, undefined, undefined, undefined,
undefined, undefined, undefined, undefined, 100]
```

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Y más novedades… los elementos guardados **no tienen por qué ser del mismo tipo,** es como si todos los elementos fuera de **tipo "mixto"**:

```
let array = [10, "Juan", false, null];
console.log(array);
```

```
[10, "Juan", false, null]
```

- Incluso podemos meter **objetos y funciones dentro del array**:

```
let array = [ {marca: "Nokia"},
              {marca: "Xiaomi"},
              function saluda() {
                console.log("Hola")
              }
            ];
console.log(array);
```

```
[[object Object] {
  marca: "Nokia"
}, [object Object] {
  marca: "Xiaomi"
}, function saluda() {
window.runnerWindow.proxyConsole.log("Hola");
}]
```



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- Además, vienen **cargados de métodos muy útiles**:
  - **push** y **pop** meten o sacan elementos del final del array (última posición).

```
let array = [ 10, 20, 30 ];

array.push(40);
array.push(50);
array.push(60);

let extraido = array.pop();

console.log(extraido, array);
```

```
60
[10, 20, 30, 40, 50]
```



  - **unshift** y **shift** meten o sacan elementos del comienzo del array.

```
let array = [ 10, 20, 30 ];

array.unshift(40);
array.unshift(50);
array.unshift(60);

let extraido = array.shift();

console.log(extraido, array);
```

```
60
[50, 40, 10, 20, 30]
```

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

- **indexOf** permite buscar la posición de un elemento automáticamente.

```
let array = [ 10, 20, 30 ];

console.log(array.indexOf(30));
console.log(array.indexOf(100));
```

```
2
-1
```

- … y hay muchos más que no vemos por el momento. Para saber más:

https://www.w3schools.com/jsref/jsref_obj_array.asp



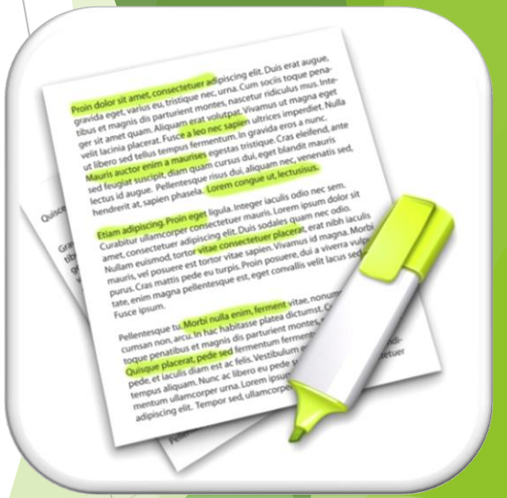
- Los arrays de JS pueden funcionar como los "clásicos arrays" de Java, pero también de forma mucho más versátil y cómoda.
- Los **arrays y los mapas de JS** son las colecciones más utilizadas en JS.

Realiza el ejercicio 10 del boletín de problemas

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
5 – Elementos básicos del lenguaje

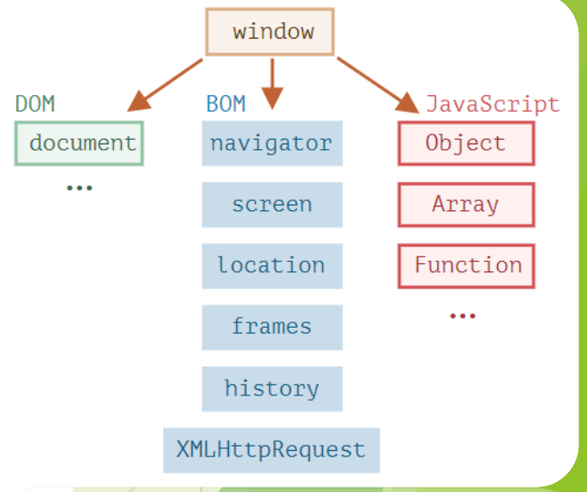
Resumen de los elementos básicos del lenguaje:

- El tipo de las variables se deduce de la asignación y puede cambiar.
- Las variables que no tienen un valor están marcadas como "undefined"
- La conversión automática de tipos puede jugar malas pasadas.
- Mejor comparar con === y !== en vez de con == y !=
- Los objetos funcionan de una forma diferente a Java. Hay que seguirlos trabajando para verles el uso real que se les da.
- Los arrays son muy versátiles y potentes. Admiten elementos de todo tipo y si te sales de los límites no dan error.



6 - El navegador como entorno de ejecución

- JS fue diseñado para ejecutarse en un entorno de ejecución concreto, el navegador. Veamos qué cosas ofrece el navegador a JS:
  - El navegador crea el objeto **window** y lo inserta en el entorno de ejecución de JS. Este objeto representa a la propia ventana del navegador y actúa como **objeto global** en el entorno de ejecución, es decir, es un **objeto accesible desde cualquier punto del código** y que contiene muchos de los métodos que ya usamos (alert, console.log…)



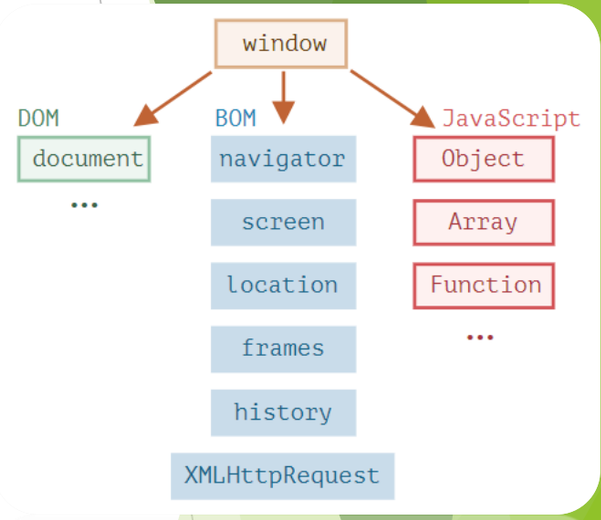

```
window.console.log("Hola");

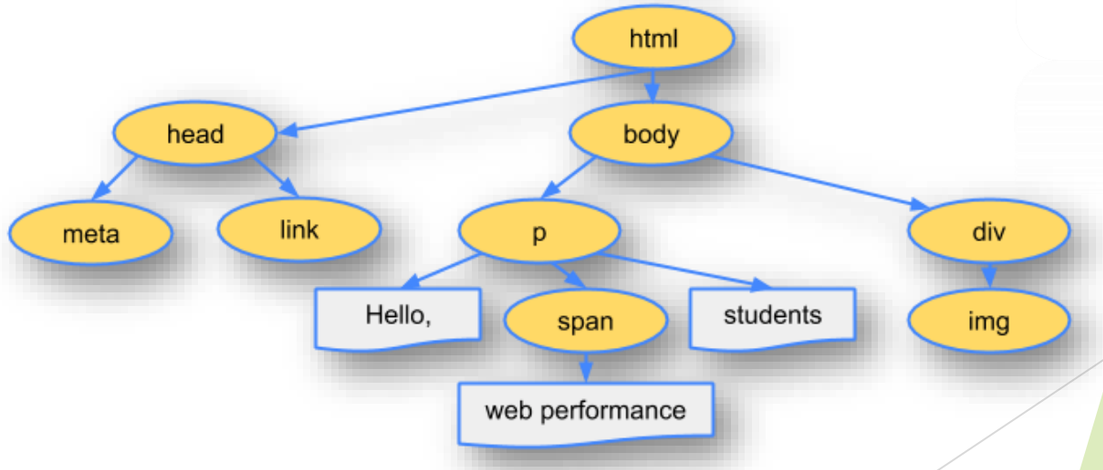
window.alert("eeeee");
```

Son métodos contenidos en window. Solo que normalmente se omite el objeto por acortar el código

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
6 – El navegador como entorno de ejecución

- El navegador también nos crea el DOM (Document Object Model) que es **una representación en árbol del documento HTML**. Cuando el navegador recibe el HTML de una página, analiza el código recibido y construye un árbol de objetos que representa al HTML fielmente.
  - Absolutamente todos los elementos del documento HTML se representan en el DOM:
    - Etiquetas
    - Atributos
    - Textos
    - Comentarios…
  - El navegador nos da acceso al DOM mediante el objeto **document**





UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
6 – El navegador como entorno de ejecución

- El navegador por último también nos da un conjunto de objetos adicionales, conocido como el **BOM (Browser Object Model)**, que funcionan como utilidades a la hora de desarrollar:
  - **Navigator** nos da información sobre el navegador usado y el sistema operativo.

```
console.log(navigator.userAgent);
console.log(navigator.platform);
```

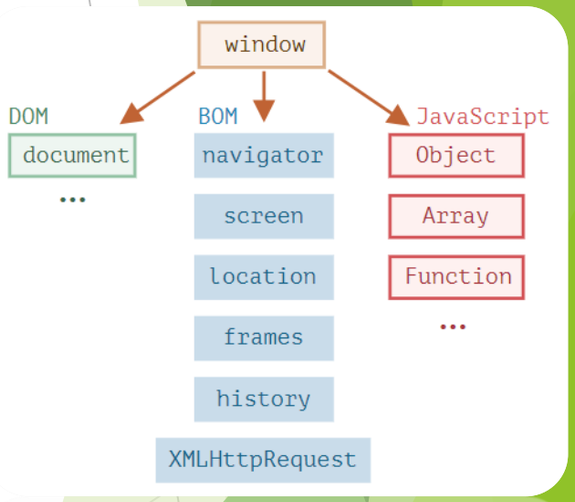
```
"Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/114.0.0.0 Safari/537.36"
"Win32"
```

  - **Location** nos dice la URL actual y permite redirigir el navegador a otra dirección nueva.

```
// muestra la URL actual
console.log(location.href);
```

```
"https://null.jsbin.com/runner"
```

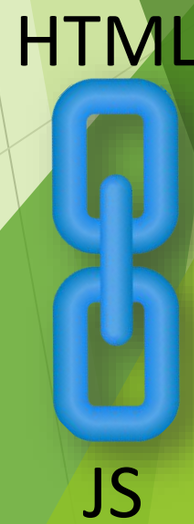
  - **XMLHttpRequest** permite hacer una petición HTTP a un servidor (abriendo la puerta a las peticiones AJAX)
  - …



7 - Integrando JS con HTML

- Desde JS podremos **manipular el DOM** de una página web pudiendo seleccionar, añadir, borrar o modificar elementos del árbol y también reaccionar ante eventos. De este modo podemos **dinamizar y enriquecer la interfaz del usuario** con menús desplegables, con validaciones automática de datos, cambios dinámicos en la estética de la web, animaciones…
- Para ello debemos **integrar nuestro código JS** con el HTML para que puedan interactuar. Tenemos dos formas de hacerlo:
  - **Incrustar el código JS directamente** en el documento HTML con la etiqueta <script>
  - **Enlazar el código JS** en el documento HTML haciendo referencia a un **fichero externo**.
- Veamos cada una de ellas.





UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
7 – Integrando JS con HTML

Incrustar el código JS directamente

- Con etiquetas <script> podemos insertar código JS en nuestro documento HTML.
- Observa el código de al lado
- Pueden aparecer tantas etiquetas <script> como se desee, pero éstas solo pueden ubicarse:
  - Dentro del <head>
  - Dentro del <body>

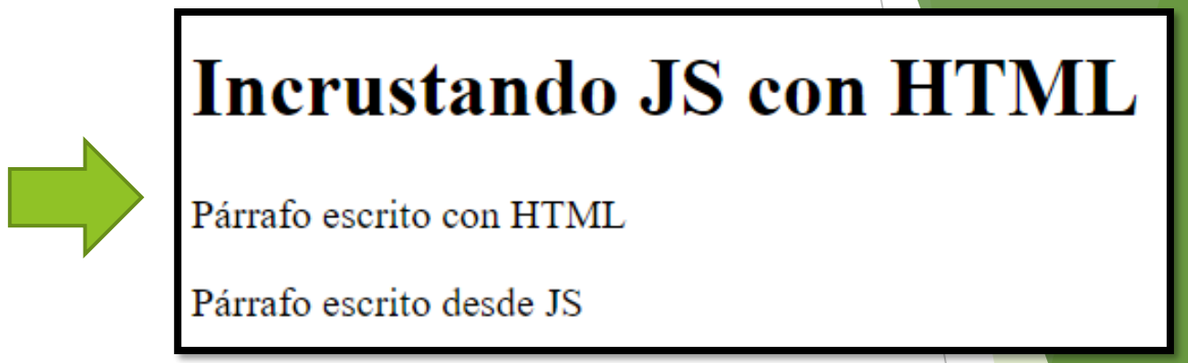
Usaremos VS Code a partir de ahora. Seguimos los pasos del documento: **"Preparación del entorno de trabajo del VS Code para JS.pdf"**

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>Incrustando JS en HTML</title>
</head>
<body>
    <h1>Incrustando JS con HTML</h1>
    <p>Párrafo escrito con HTML</p>
    <script>
      document.write("<p>Párrafo escrito desde JS</p>");
      console.log("Esto va a la consola de JS");
    </script>
</body>
</html>
```

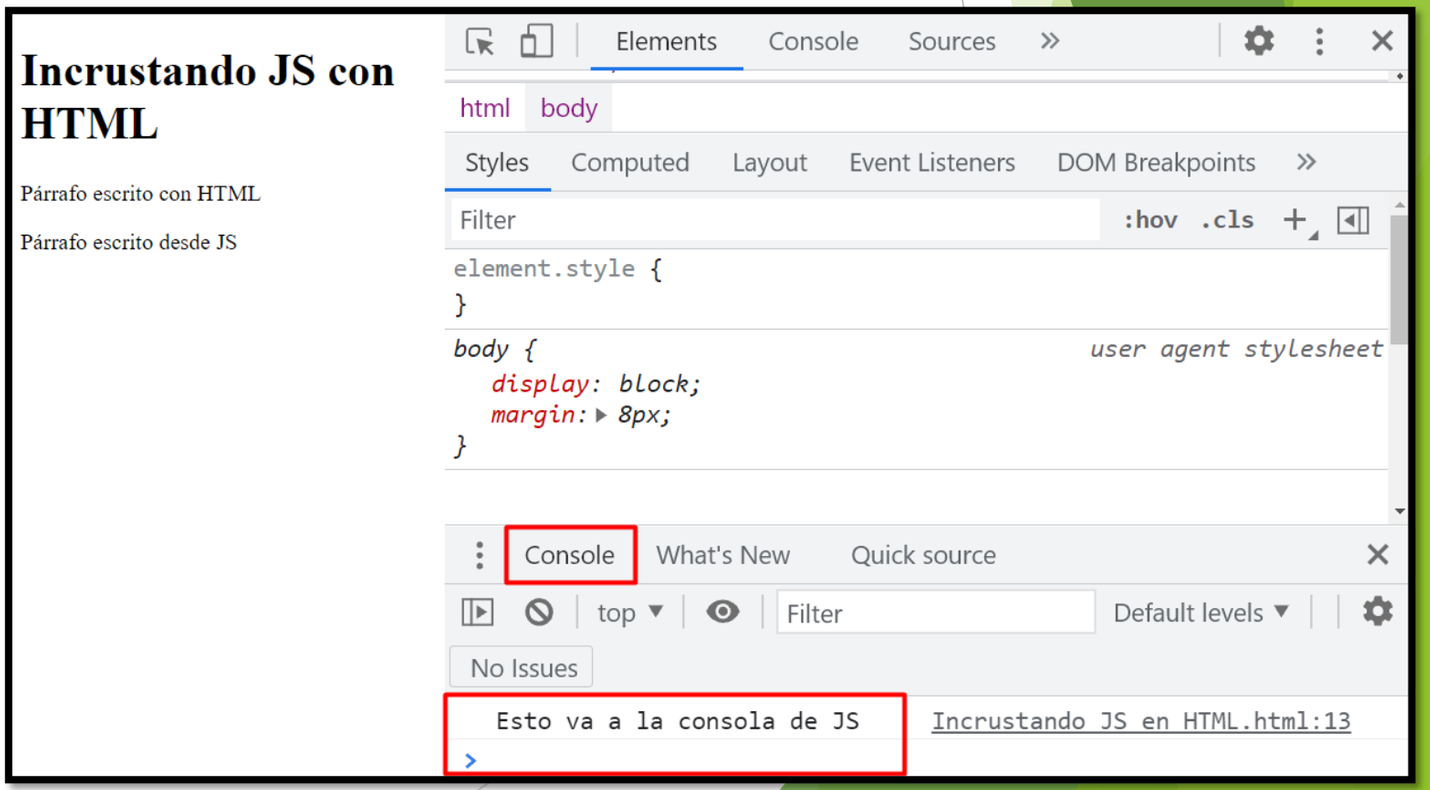
UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS

7 – Integrando JS con HTML

- En un navegador, el código anterior se vería así. Como vemos, **el código JS se ejecuta exactamente en el lugar del documento HTML en el que se había insertado la etiqueta <script>.**



- Además, nuestro navegador tiene una **consola de JS integrada** que se muestra pulsando **F12** (en el caso de las DevTools de Chrome)



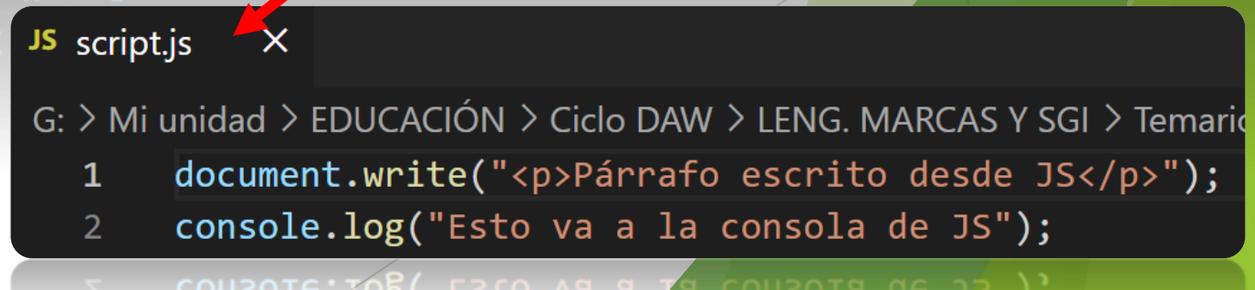
UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
7 – Integrando JS con HTML

Enlazar el código JS

- En este caso la etiqueta <script> lleva un atributo **src** en el que se indica la ruta y el fichero que contiene el script.
- Cuando el navegador se encuentra esta etiqueta, busca el fichero asociado y después **copia el código sobre el documento HTML y lo ejecuta**.

OJO: como se hace una COPIA del código del fichero enlazado, la sintaxis de "autoclausura" NO FUNCIONA <script src="..." />

```html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>Enlazando JS en HTML</title>
</head>
<body>
    <h1>Enlazando JS con HTML</h1>
    <p>Párrafo escrito con HTML</p>
    <script src="script.js"></script>
</body>
</html>
```



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
7 – Integrando JS con HTML

- El resultado de la ejecución de este ejemplo es idéntico al del caso anterior. De hecho, se está haciendo una COPIA del contenido del fichero JS de modo que el documento HTML resultante queda exactamente igual que cuando incluíamos el código directamente.
- Este mecanismo **acelera la carga de los sitios web en el navegador** ya que los ficheros de JS se quedan guardados en la **memoria caché** y, si hay varias páginas de un mismo sitio web que utilizan los mismos ficheros de script, entonces solo habrá que descargarlos una vez.
- **¿Y dónde es mejor poner las etiquetas <script>? Pues depende…**
  - Si el script no necesita que el DOM esté construido (scripts de terceros: anuncios, iconos (fontawesome), jQuery…), entonces AL PRINCIPIO.
  - Si el script necesita que se haya construido el DOM completamente para poder acceder a sus elementos, entonces AL FINAL. **Esta será la opción preferida en este curso…**





Realiza los ejercicios del 11 al 15 del boletín de problemas

8 - Selección de elementos del DOM con JS

- Hay varias formas de seleccionar nodos del DOM con JS. Vamos a estudiar primero las que más se usan y después nombraremos las que menos, pero que todavía suelen aparecer en código antiguo.

Selección mediante selectores CSS

- Con **document.querySelector("selectorCSS")** podemos obtener el **primer nodo que encaje** con el selector del parámetro. Ejemplos:
  - **document.querySelector("p")** nos devuelve el primer nodo que se corresponde con una etiqueta <p> del documento HTML.
  - **document.querySelector(".nav-bar")** nos devuelve el primer nodo del documento que tenga el atributo class="nav-bar"
  - **document.querySelector("#footer")** nos devuelve el primer nodo del documento que tenga por id="footer"



Realiza el ejercicio 16 del boletín de problemas

Refresca tus habilidades de selección con CSS: **https://flukeout.github.io/**

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
8 - Selección de elementos del DOM con JS

- Si no se encuentra **ninguna ocurrencia** para el selector buscado el método **querySelector** devuelve **null**.
- Por otro lado, con **document.querySelectorAll("selectorCSS")** obtenemos un array con **todos los nodos que encajen** con el selector CSS especificado.
- Ejemplo:

```
<script>
    let arrayParrafos = document.querySelectorAll("p");
    console.log(arrayParrafos);
</script>
```



Se imprime en consola el array de nodos <p> del documento

Si no se encuentra ninguna ocurrencia entonces **querySelectorAll** devuelve el **array vacío []**

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
8 - Selección de elementos del DOM con JS

- También podemos recorrer el array de nodos como haríamos con cualquier otro array. Observa:

```
<script>
    let arrayParrafos = document.querySelectorAll("p");
    for(let parrafo of arrayParrafos) {
        console.log(parrafo);
    }
</script>
```



- Si lo piensas bien, las siguientes llamadas son equivalentes:

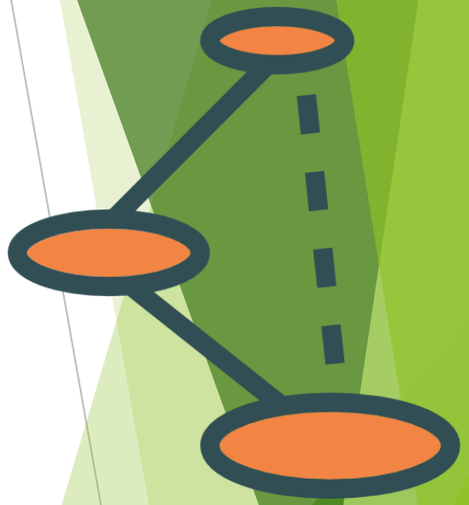
querySelector("x") = **querySelectorAll**("x")[0]

Realiza el ejercicio 17 del boletín de problemas

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
8 - Selección de elementos del DOM con JS

Atajos a elementos de uso frecuente

- Las etiquetas de uso cotidiano como <html>, <head> y <body> tienen su abreviatura de forma que podamos acceder a ellas directamente.
- Para ello, el objeto **document** define tres propiedades que nos devuelve directamente los nodos correspondientes del DOM:
  - document.**documentElement** devuelve el nodo <html>
  - document.**head** devuelve el nodo <head>
  - document.**body** devuelve el nodo <body>
- **OJO:** si usamos **document.body** en un script que hemos **insertado en el head no funcionará** y nos devolverá **null**. Esto es porque el DOM se construye de "arriba-abajo" y los scripts se ejecutan a medida que van apareciendo en el código. Como dijimos anteriormente, las etiquetas <script> mejor al final…





UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
8 - Selección de elementos del DOM con JS

Selección mediante métodos getElementByXXX

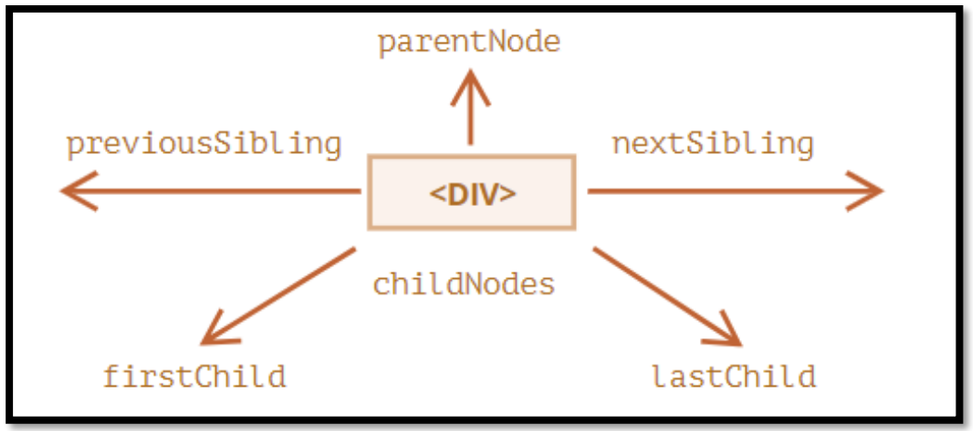
- Los métodos querySelector y querySelectorAll nos permiten pensar **"en modo CSS"** haciendo más sencillo el trabajo de selección de elementos. Antes de estos métodos se utilizaban un conjunto de métodos más largos de escribir y que usaban un método con nombre diferente para cada caso. Estos son:
  - **getElementById("footer")** devuelve el elemento con id="footer" del documento.
  - **getElementsByTagName("p")** devuelve un array con todos los elementos etiquetados con <p> del documento.
  - **getElementsByClassName("container")** devuelve un array con todos los elementos cuya clase sea "container".
- Todavía se usan y os los podéis encontrar…



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
8 - Selección de elementos del DOM con JS

Selección por relación de parentesco

- Otra forma de seleccionar elementos del DOM es mediante la relación de parentesco entre un nodo dado y otro buscado. JS proporciona funciones para obtener el **nodo padre, hermano o hijo** de un nodo dado. Observa la siguiente imagen:





- Este enfoque **es menos habitual** y no lo estudiaremos en profundidad.
- Para saber más: https://es.javascript.info/dom-navigation#en-la-parte-superior-documentelement-y-body

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
8 - Selección de elementos del DOM con JS



Haciéndonos la vida un poco más fácil con debug-utils.js

- En este curso trabajaremos con **Vanilla JS** (JavaScript puro, sin librerías). Como JS es demasiado "flexible", muchos errores **no aparecen donde se originan**, sino más tarde, con mensajes confusos, dificultando la localización y arreglo del error.
- Para programar **con menos sufrimiento** vamos a seguir la **filosofía Fail Fast** (fallar pronto) en la que, si algo está mal, **queremos que "reviente" inmediatamente y con un mensaje claro.**
- Para ello usaremos un "kit de herramientas", un archivo de apoyo llamado **debug-utils.js** que incluiremos en el <head> del documento html del añadiendo la siguiente línea:

```
<script src="debug-utils.js"></script>
```

Descarga **debug-utils.js** y **debug-utils-example.html** y analiza ambos ficheros

9 – Manipulación de los nodos del DOM con JS

- Una vez hemos aprendido a seleccionar elementos, podemos modificar el valor de sus propiedades y veremos los cambios reflejados en la página HTML. Veamos algunas de ellas:

innerText

- Esta propiedad permite cambiar el contenido de texto encerrado entre dos etiquetas.



Abre y estudia el fichero **innerText.html**

- La función **setTimeout** permite establecer una cuenta atrás de 2 segundos (2000 milisegundos) y cuando se cumpla el tiempo se invoca a la función **cambiar**.

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

innerHTML

- Esta propiedad permite cambiar el contenido HTML encerrado en un nodo. Es una propiedad muy utilizada ya que permite añadir o quitar contenido rápidamente a una página web.

Abre y estudia el fichero **innerHTML.html**



- Observa el uso de las comillas en:

```
div.innerHTML ='<a href="https://www.wikipedia.org/">Wikipedia</a>'
```

Dado que el texto asignado contiene **comillas dobles** en el atributo **href**, encerramos todo el texto entre **comillas simples**. También funcionaría si lo hacemos al revés (primero dobles y luego simples).

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

innerHTML VS innerText

- Para entender mejor la diferencia entre ambas propiedades retoca el código del ejemplo **innerHTML.html cambiando las asignaciones a la propiedad innerHTML por innerText** y observa el resultado:

```
Cada segundo vamos a publicar un nuevo contenido

<p>¡Ey! Este párrafo antes no estaba...</p>
<a href="https://www.wikipedia.org/">Wikipedia</a>
<p>Ciudades a visitar</p><ul><li>Buenos Aires</li><li>Roma</li><li>Sídney</li></ul>
```



- **innerText** pinta el texto tal cual, **SIN interpretar las etiquetas** que se pudiera encontrar.
- Sin embargo, **innerHTML** trabaja con **fragmentos HTML** que **SÍ SERÁN INTERPRETADOS** por el navegador.

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

Legibilidad y versatilidad con las template strings

- Si nuestro contenido HTML es muy largo y queremos que se lea algo mejor, tenemos que hacer algo así:

```
"<ul>"+
    "<li>Buenos Aires</li>"+
    "<li>Roma</li>"+
    "<li>Sídney</li>"+
"</ul>";
```

- Esto **no es muy cómodo**… así que JS nos ofrece una herramienta MUY ÚTIL que son las **template strings** o "cadenas plantilla". Veamos un ejemplo:

Abre y estudia el fichero **templateStrings.html**



- El acento grave o **"backtick"** se obtiene pulsando la tecla de la derecha de la P y después la barra espaciadora.
- Además de poder escribir textos de varias líneas cómodamente, podemos usar expresiones **${ expr }** que se evaluarán en tiempo de ejecución. **¡Súper útil!**

Realiza los ejercicios 18-21 del boletín de problemas

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

Los atributos se convierten en propiedades automáticamente

- Sabemos que cada etiqueta HTML tiene su propio conjunto de atributos:

```
<img src="captura.png" alt="Captura de pantalla"/>
<a href="https://wikipedia.org" target="_blank">wiki</a>
<label for="nombre">Dime tu nombre: </label>
<input id="nombre" type="text"/>
```

- Cuando se crea el DOM todos los atributos de una etiqueta **se convierten automáticamente en propiedades con el mismo nombre**. De este modo, siguiendo con el ejemplo anterior, podríamos escribir:

```
let link = document.querySelector("a");
link.href = "https://google.com";
link.target = "_self";
link.innerText = "Google";
```

```
let img = document.querySelector("img");
img.src = "default.jpg";
img.alt = "Texto por defecto";
```

Realiza los ejercicios 22-23 del boletín de problemas



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

Modificando las clases CSS

- Estilizar/maquetar elementos del HTML pasa por realizar dos tareas:
  - **En el HTML**: identificar al elemento que queremos tratar, usualmente mediante el uso de atributos **class** o **id**.
  - **En el CSS**: seleccionar el elemento identificado y añadirle propiedades CSS.
- Veamos cómo podemos estilizar/maquetar dinámicamente usando JS:

Abre y estudia el fichero **classList.html**

- La propiedad **classList** de un elemento nos permite acceder su la lista de clases CSS para poder modificarla cómodamente con los métodos **add, remove o toggle** (alternar).



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

- Además, la propiedad **classList** nos ofrece otros métodos de interés:
  - **contains ("clase")** devuelve true si la lista de clases contiene la que se pasa como parámetro.
  - **replace ("clase", "claseNueva")** reemplaza la "clase", si existe, por la "claseNueva".
- También podemos **eliminar todos los valores de la lista de clases** asignando la cadena vacía a la propiedad classList:

```
let h1 = document.querySelector("h1");
h1.classList = "";
```

- Y añadir varias clases en una sola llamada al método **add**:

```
h1.classList.add("rojo", "centrar", "grande");
```



Realiza los ejercicios 24-26 del boletín de problemas

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

Propiedades old-school: className y style

- Antes de que apareciera la propiedad **classList** se usaba:
  - **La propiedad className**: representa a TODA LA CADENA del atributo "class" de HTML. Esto es bastante incómodo porque necesitamos tratar toda la cadena de texto para añadir o quitar clases individualmente. Observa:

Abre y estudia el fichero **className.html**

NO debes usar este tipo de código, PERO todavía te lo puedes encontrar en páginas antiguas…



  - **La propiedad style**: representa al atributo **style** de HTML que permite asignar propiedades CSS directamente a un elemento (no lo estudiamos).

```
h1.style.textAlign = "center";
h1.style.color = "red";
```

Este enfoque viola el principio de **separación de responsabilidades** (los scripts empezarían a hacer tareas propias del CSS). No se recomienda.

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

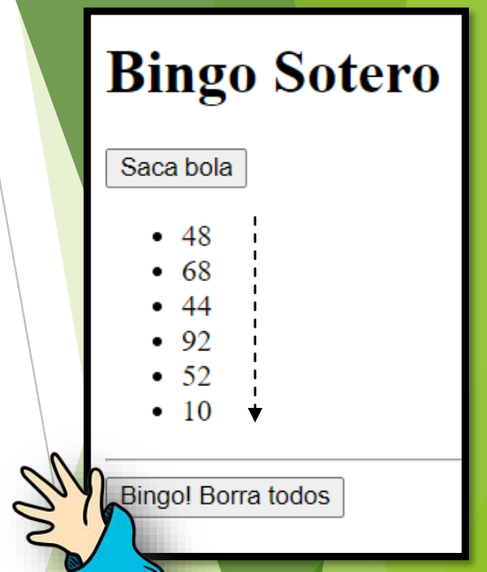
Insertar y eliminar nodos del DOM

- Ahora queremos **"vestir" o "desvestir" nuestro DOM**, añadiendo nuevos nodos o quitando otros "sobrantes". Veamos cómo se hace:

Abre y estudia el fichero bingo-sotero.html

DEBES SABER que cada vez que pinchamos un botón se llama a la función indicada en el atributo ***onclick***

- Observa que para añadir un nuevo nodo al árbol lo podemos hacer de dos formas (elige la que más sencilla te parezca):
  - **Usando createElement y appendChild.**
  - **Usando fragmentos HTML e innerHTML**



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

Con createElement y appendChild

- Observa que, en este caso, para añadir un nuevo nodo al árbol lo hacemos en 3 pasos:
  - **Creamos un nodo "desconectado"** del DOM con **createElement.** Ejemplo: `let nuevoNodo = document.createElement("li");`
  - **Lo "rellenamos"**: le ponemos texto con innerText… o atributos... Ejemplo: `nuevoNodo.innerText = numero;`
  - **Lo "conectamos" al DOM** desde su nodo padre con el método **appendChild**. Ejemplo: `let ul = qs(".lista-numeros");`

    `ul.appendChild(nuevoNodo);`
- Por otro lado, para borrar un nodo tenemos que llamar al método **removeChild** desde el nodo padre.



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

- Los nodos del DOM poseen un conjunto de **propiedades y métodos versátiles** que nos ayuda en esto de añadir/quitar nodos. Destacamos:
  - La propiedad **children** es un array que contiene los nodos hijos del nodo desde el que usa.
  - El método **cloneNode()** devuelve un duplicado del nodo desde el que se invoca.
  - El método **replaceChild (nodoHijoNuevo, nodoHijoAntiguo)** reemplaza el nodo hijo antiguo por el nuevo
  - El método **insertBefore (nodoHijoNuevo, nodoHijoReferencia)** inserta el nodo nuevo antes del nodo que se toma como referencia. Si el nodo de referencia es null entonces se insertará en último lugar…

¿Qué crees que pasaría en el "Bingo Sotero" si sustituimos la línea del *appendChild* por las siguientes?

```
let lista = document.querySelector(".lista-numeros");
lista.insertBefore(li, lista.children[0]);
```



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

Con fragmentos HTML e innerHTML

En este caso, para añadir un nuevo nodo al árbol lo hacemos así:

- **Creamos un fragmento HTML "desconectado" y lo "rellenamos"**. Ejemplo:
  ```
  let fragmentoHTML = `<li>${numero}</li>`
  ```
- **Lo "conectamos" a su nodo padre usando innerHTML**. Ejemplo:
  ```
  let lista = qs(".lista-numeros");

  lista.innerHTML += fragmentoHTML;
  ```

- Por otro lado, si necesitamos borrar el contenido de un nodo podemos asignar la cadena vacía a su **innerHTML**. Ejemplo:

```
ul = qs("ul");

ul.innerHTML = "";
```

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
9 – Manipulación de los nodos del DOM con JS

- Si combinamos la **creación de nodos/fragmentos, los template strings y la información almacenada en objetos** conseguimos mucha versatilidad y potencia expresiva. Para entenderlo mejor vamos a **remodelar el método *cantaBingo***:

Abre y estudia el fichero
bingo-sotero-ganadores.html



- La parte que más nos interesa de este ejemplo está en el método ***publicaGanador***, concretamente las siguientes líneas:

```
let fragmento = `<tr><td>${ganador.time}</td>
                 <td>${ganador.name}</td>
                 <td>${ganador.address}</td></tr>`;

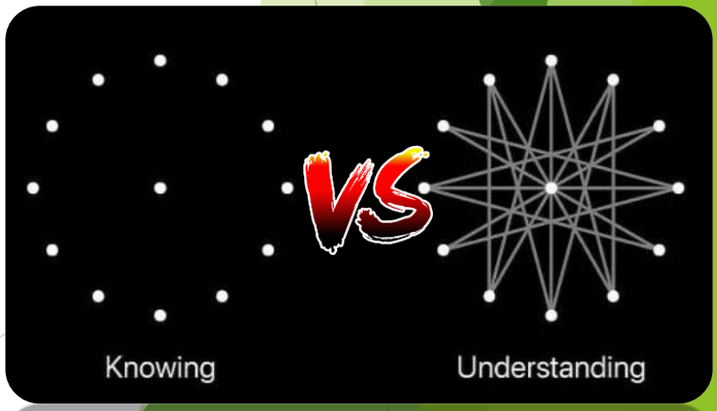
let tabla = document.querySelector("table");
tabla.innerHTML += fragmento;
```

Realiza los ejercicios 27-29 del boletín de problemas

10 – Vanilla JS vs Frameworks/Librerías

- En esta unidad hemos sentado las **bases del manejo de páginas web con JS "puro" o Vanilla JS**, es decir, sin usar librerías o frameworks.
- Sin embargo, existen **librerías** (como jQuery) o **frameworks** (como React, Angular…) que añaden funcionalidad al lenguaje y nos permiten hacer más cosas con menos código. Este tipo de código más sencillo y potente lo estudiareis en 2º curso.
- Sin embargo, aunque en la práctica se usan librerías/frameworks, conocer los fundamentos de JS y su funcionamiento interno te hará un mejor desarrollador/a. No basta solo con "**saber hacer cosas**", tienes que "**comprender lo que haces**".





UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
Cierre de la unidad

Cierre de la unidad

- En esta unidad hemos presentado JS como un lenguaje de programación un "tanto peculiar" y lo hemos utilizado para **dinamizar y añadir funcionalidad a nuestros documentos web.**
- Hemos presentado las **bases de la programación en entorno de front-end**, pero hay muchas cosas que se han quedado en el tintero y que trataréis en 2º (arrow functions, eventos, async functions, destructuring, modules…)
- Para saber más:
  - Javascript Moderno: https://es.javascript.info/
  - W3Schools: https://www.w3schools.com/js/default.asp
  - Uniwebsidad: https://uniwebsidad.com/libros/javascript

Saber más



Anexo – Depurando JS desde el navegador

- Como habrás comprobado detectar errores en JS no es fácil. El simple hecho de seleccionar elementos del DOM ya es algo **propenso a errores semánticos difíciles de detectar**.
- Observa:
  - **document.querySelector("nav-bar")** devuelve **null** si lo que queremos es seleccionar un elemento ** …** Faltaría el .
  - **document.querySelector("#footer")** devuelve **null** si lo que queremos es seleccionar un elemento **<footer>…** Sobraría el #
- Estas expresiones fallarán en tiempo de ejecución, nuestro código no funcionará y no sabremos el motivo. En el mejor de los casos obtendremos un mensaje de error en la consola de JS.



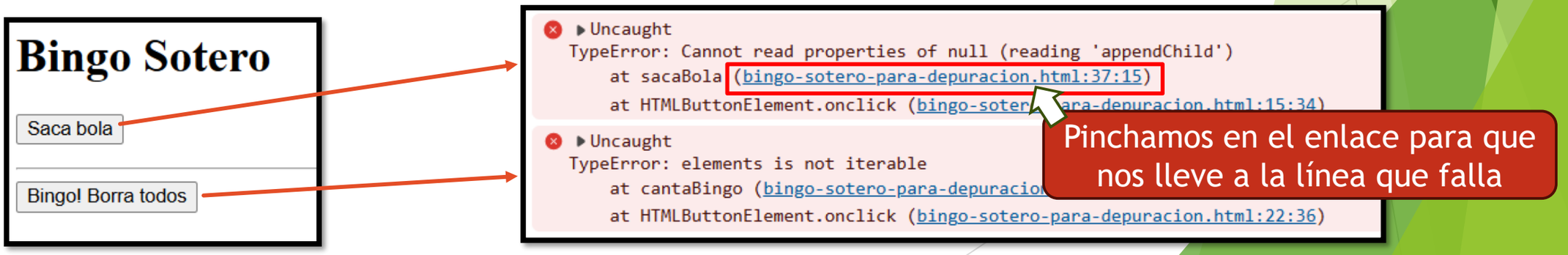
UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
Anexo – Depurando JS desde el navegador

- Para localizar este tipo de errores muchas veces será **necesario DEPURAR el código**. Veamos cómo se hace desde el navegador…

Hacemos la prueba y depuración del fichero bingo-sotero-para-depuracion.html

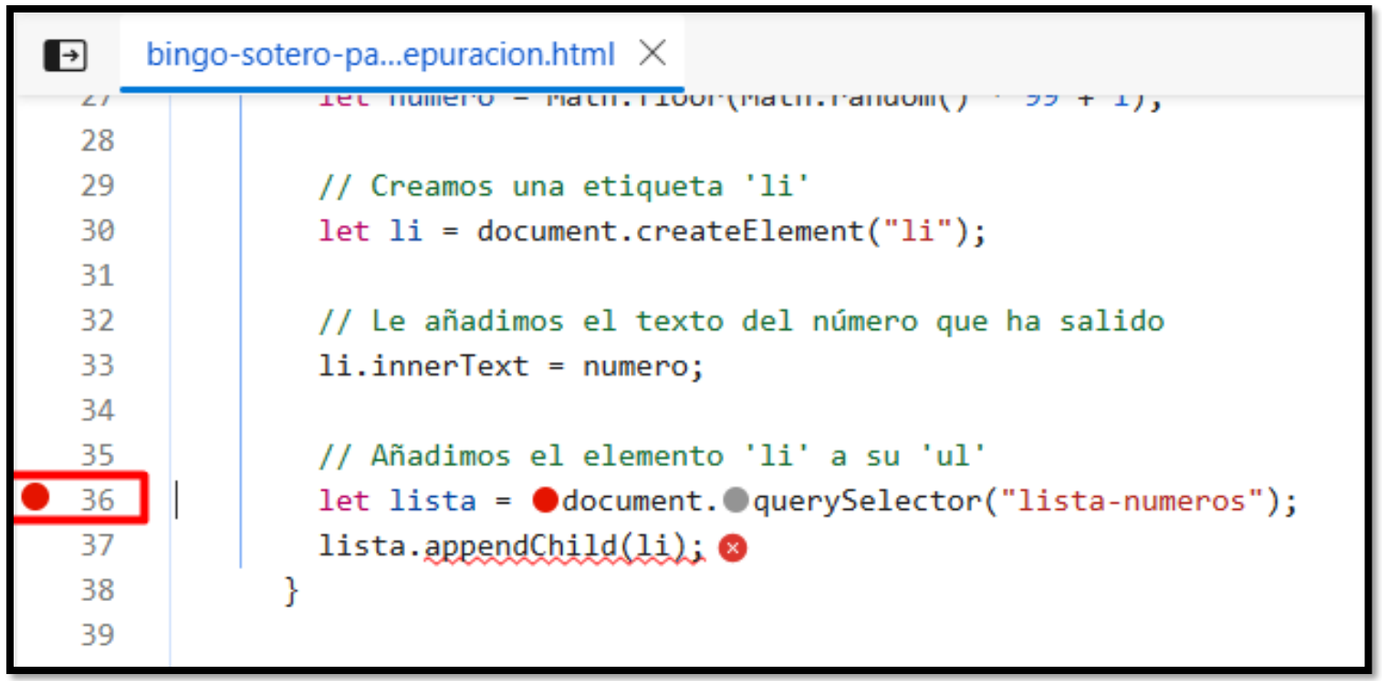
NOTA: las siguientes imágenes son del Microsoft Edge, pero el procedimiento es prácticamente el mismo con otro navegador

- Usaremos el **Live Server** para abrir nuestra página web. Una vez se cargue en el navegador, pulsaremos F12 para abrir la ventana de **"herramientas de desarrollador"**
- Pulsamos los botones y vemos que aparecen errores en la consola:



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
Anexo – Depurando JS desde el navegador

- Si no somos capaces de localizar el error con la con la información que nos da la consola nos tocará **ejecutar paso a paso el código**:
  - Para ello marcaremos con un **punto de ruptura** (breakpoint) la línea que da problemas o alguna de las anteriores.





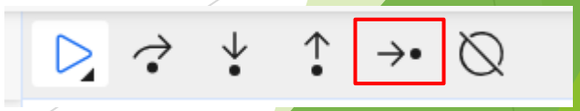
Ahora volvemos a pulsar el botón para disparar el proceso, pero ahora la ejecución se detendrá en nuestro breakpoint

UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
Anexo – Depurando JS desde el navegador

- Nuestro código se habrá pausado en la línea del breakpoint. Además, tendremos visibles los valores de las expresiones ya evaluadas:

```
25      function sacaBola() {
26        // Sacamos aleatorio un número del 1 al 99
27        let numero = Math.floor(Math.random() * 99 + 1);   numero = 5
28
29        // Creamos una etiqueta 'li'
30        let li = document.createElement("li");   li = li {value: 0, type: '', title: '', lang: '', translate: true, …}
31
32        // Le añadimos el texto del número que ha salido
33        li.innerText = numero;   li = li {value: 0, type: '', title: '', lang: '', translate: true, …}   numero = 5
34
35        // Añadimos el elemento 'li' a su 'ul'
36        let lista = document.querySelector("lista-numeros");
37        lista.appendChild(li);
38      }
```

- Ahora podremos usar los controles para avanzar paso a paso:



UD 4 – MANIPULACIÓN DE DOCUMENTOS WEB CON JS
Anexo – Depurando JS desde el navegador

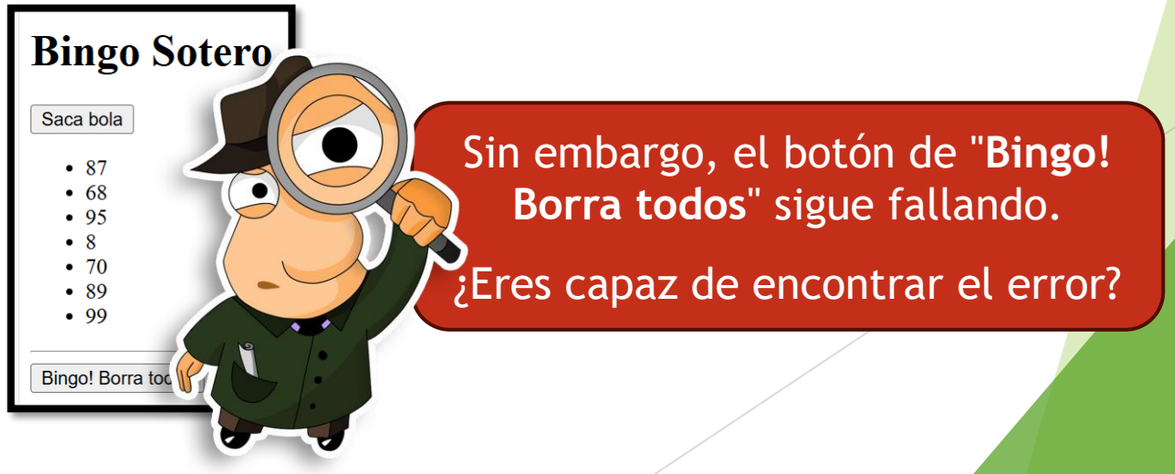
- Al avanzar observamos que la variable "**lista**" se carga con **null**. Esto es lo que provoca el fallo en la siguiente línea.

```
// Añadimos el elemento 'li' a su 'ul'
let lista = document.querySelector("lista-numeros");   lista = null
lista.appendChild(li);
```

- querySelector falla en la búsqueda del elemento porque "**lista-numeros**" es una clase y la expresión correcta debería ser "**.lista-numeros**"

- **¡Ya hemos encontrado uno!** Volvemos al VS Code, lo arreglamos y volvemos a ejecutar.

Ahora ya funciona el botón "**Saca bola**"



Sin embargo, el botón de "**Bingo! Borra todos**" sigue fallando.

¿Eres capaz de encontrar el error?


The End